

EXPLAINABLE MULTI-ATTRIBUTE MANGO QUALITY GRADING USING MOBILENETV3-LARGE

Draft Thesis Report

Dayarathna W.S.R

(IT22257154)

BSc (Hons) degree in Information Technology
Specializing in Information Technology

Department of Information Technology

Sri Lanka Institute of Information Technology

Sri Lanka

April 2026

DECLARATION

I declare that this is my own work and this dissertation does not incorporate without acknowledgement any material previously submitted for a Degree or Diploma in any other University or institute of higher learning and to the best of my knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgement is made in the text.

Also, I hereby grant to Sri Lanka Institute of Information Technology, the non-exclusive right to reproduce and distribute my dissertation, in whole or in part in print, electronic or other medium. I retain the right to use this content in whole or part in future works such as articles or books.

Signature: _____ Date: _____

The above candidate has carried out research for the bachelor's degree dissertation under my supervision.

Signature of the supervisor (Ms. Karthiga Rajendran):

_____ Date: _____

Signature of the co-supervisor (Ms. Poojani Gunathilake):

_____ Date: _____

ABSTRACT

This individual summary report presents the mango quality grading component of the FreshRoute smart fruit supply chain project. The original grading prototype used a pretrained MobileNetV3-large model to classify mango images into Grade A, Grade B, and Grade C. Although that design was useful as a baseline, it did not clearly explain which visible fruit features guided the decision. To solve that issue, this report redesigns the component into an explainable multi-attribute grading system. The updated dataset keeps the three grade folders but adds a CSV label file with color stage, spot severity, blemish severity, and optional defect flags. The preprocessing flow uses background cleanup, crop refinement, resizing, augmentation, and normalization. The model uses a shared MobileNetV3-large backbone with four output heads for color stage, spot severity, blemish severity, and final grade. A direct grade prediction path is combined with a rule-based path derived from the attribute outputs. This allows the system to stay practical while also becoming easier to explain. The report describes the full methodology, commercialization idea, testing plan, implementation flow, and discussion of strengths and limitations. The main contribution of this component is the shift from a black-box grade-only classifier to a clearer grading module that is better suited for thesis defense and future integration into the full quality verification system.

Keywords: mango grading, explainable AI, MobileNetV3, transfer learning, fruit quality verification

DEDICATION

This report is dedicated to my parents, teachers, supervisors, and everyone who encouraged me to keep learning and improving during this research journey.

ACKNOWLEDGEMENT

I would like to thank my supervisor, Ms. Karthiga Rajendran, for her guidance, support, and helpful feedback throughout this component. I also thank my co-supervisor, Ms. Poojani Gunathilake, for her advice and for helping me improve the technical and writing quality of this work.

I am grateful to the evaluation panel for their comments during earlier reviews. Their questions helped me understand where the first grading approach was weak and why the system needed a clearer explanation of the grading features. That feedback directly influenced the updated dataset design and the new multi-attribute model described in this report.

I also thank my group members in the wider FreshRoute project for their support and cooperation. Even though this report focuses on my individual grading component, the full project environment helped me understand how this component fits into a larger smart fruit supply chain system.

Finally, I would like to thank my family and friends for their patience, encouragement, and motivation. Their support helped me complete this research report with confidence.

TABLE OF CONTENTS

<i>DECLARATION</i>	<i>i</i>
<i>ABSTRACT</i>	<i>ii</i>
<i>DEDICATION</i>	<i>iii</i>
<i>ACKNOWLEDGEMENT</i>	<i>iv</i>
<i>TABLE OF CONTENTS</i>	<i>v</i>
<i>LIST OF FIGURES</i>	<i>viii</i>
<i>LIST OF TABLES</i>	<i>ix</i>
<i>LIST OF ABBREVIATIONS</i>	<i>x</i>
<i>LIST OF APPENDICES</i>	<i>xi</i>
1. INTRODUCTION	1
1.1 Background and literature survey	1
1.2 Literature survey	3
1.3 Research gap	6
1.4 Research problem	7
1.5 Research objectives	8
2. METHODOLOGY	10
2.1 Methodology overview	10
2.2 Scope of the individual component	10
2.3 Dataset definition and annotation rubric	11
2.4 Dataset split and file organization	13
2.5 Preprocessing and fruit isolation	15
2.6 Classical support features	17
2.7 Deep learning model design	18
2.8 Final grade decision logic	20
2.9 Training strategy	22
2.10 Commercialization aspect of the product	24
2.11 Testing and implementation	26
3. RESULTS AND DISCUSSION	29
3.1 Results	29
3.2 Research findings	31
3.3 Discussion	32
3.4 Summary of each student's contribution	34
4. CONCLUSION	36
REFERENCES	38
GLOSSARY	40

APPENDICES.....	41
Appendix A Annotation rubric.....	41
Appendix B CSV format.....	42
Appendix C Folder structure and outputs.....	42
Appendix D Training settings and implementation notes	43

EXPLAINABLE MULTI-ATTRIBUTE MANGO QUALITY GRADING USING MOBILENETV3-LARGE

Draft Thesis Report

Dayarathna W.S.R

(IT22257154)

BSc (Hons) degree in Information Technology
Specializing in Information Technology

Department of Information Technology

Sri Lanka Institute of Information Technology

Sri Lanka

April 2026

LIST OF FIGURES

Figure 1: Scope of the individual grading component within the larger research project	3
Figure 2: Coverage of key system needs in earlier studies and in the proposed component	6
Figure 3: Dataset redesign and annotation flow for the grading component.....	13
Figure 4: Stratified dataset split used in the baseline and upgraded model.....	14
Figure 5: Image preprocessing pipeline used before feature learning.....	16
Figure 6: Explainable multi-attribute mango quality grading architecture	20
Figure 7: Simple rule-based grade mapping from the three attribute heads	21
Figure 8: Training, validation, and report generation workflow	23
Figure 9: Combined loss weights used for multi-head learning.....	24
Figure 10: Main output files that are saved after training and evaluation	24
Figure 11: Possible commercialization path for the grading module	25
Figure 12: Two grade outputs used for comparison in the proposed system.....	33
Figure 13: Explainability flow used to generate Grad-CAM visual evidence	34
Figure 14: Summary view of strengths and present limitations of the component	34

LIST OF TABLES

Table 1: LIST OF ABBREVIATIONS	x
Table 2: Existing research methods relevant to fruit grading.....	5
Table 3: Research gap comparison between earlier studies and the proposed component	5
Table 4: Summary of the individual grading component scope	9
Table 5: CSV label structure used in the updated grading dataset.....	12
Table 6: Final grade rubric used in the component	12
Table 7: Attribute rubric for explainable grading labels	13
Table 8: Recommended file organization for the individual component	15
Table 9: Preprocessing operations used in the grading pipeline.....	17
Table 10: Classical support features and their use in the system.....	18
Table 11: Outputs of the multi-head MobileNetV3-large model.....	19
Table 12: First practical rule set used for final grade mapping.....	21
Table 13: Main hyperparameters and training settings drawn from the baseline code	23
Table 14: Commercialization opportunities and key concerns	26
Table 15: Testing plan for the grading component.....	27
Table 16: Software and hardware environment for the component.....	28
Table 17: Main output files produced by the grading pipeline	31
Table 18: Research findings summary.....	32
Table 19: Evaluation measures and how they should be interpreted	33
Table 20: Summary of the contribution of this student	35
Table 21: Key conclusions and future improvement directions	36
Table 22: Glossary of key terms used in the report	40
Table 23: Detailed annotation rubric checklist.....	41
Table 24: Loss components in the multi-head model	43

LIST OF ABBREVIATIONS

Table 1: LIST OF ABBREVIATIONS

Abbreviation	Description
AI	Artificial Intelligence
CNN	Convolutional Neural Network
CSV	Comma-Separated Values
GLCM	Gray Level Co-occurrence Matrix
GPU	Graphics Processing Unit
HSV	Hue, Saturation, Value
ONNX	Open Neural Network Exchange
RGB	Red, Green, Blue
SVM	Support Vector Machine
Grad-CAM	Gradient-weighted Class Activation Mapping

LIST OF APPENDICES

Appendix A Annotation rubric	41
Appendix B CSV format.....	42
Appendix C Folder structure and outputs	42
Appendix D Training settings and implementation notes.....	43
Appendix E Mermaid architecture code	43
Appendix F Evaluation checklist	44

1. INTRODUCTION

1.1 Background and literature survey

Fruit quality matters a lot in the fresh fruit trade. Buyers expect fruits that look good and match what was promised. Sellers also expect a fair system that judges their produce in a consistent way. In Sri Lanka, many fruit transactions still depend on quick visual checks done by people. This method is common, but it is not always reliable. Two people can look at the same fruit and give different opinions. Because of this, manual grading often creates confusion, disagreement, and loss of trust between the people involved in the supply chain.

Mango is a good fruit for image-based quality grading because many of the visible quality signs appear on the peel. Color changes give an early idea about ripeness. Small dark spots can suggest mild surface damage or disease signs. Blemishes, scratches, and rough marks can reduce the market value even when the fruit is still edible. In normal marketplaces, these visible signs are used by sellers and buyers in a practical way. This makes mango a suitable starting point for a computer vision study that focuses on visible appearance rather than internal quality.

The wider group project, FreshRoute, was designed to support fruit quality verification and complaint validation in a smart supply chain. My individual component focuses on the quality grading part only. This is the part that looks at a single mango image and predicts the visible quality level. The purpose of this report is to describe that component in a detailed and clear way. The report explains why the component is needed, how the dataset is organized, how the new grading labels are defined, how MobileNetV3-large is used, and how the model becomes more explainable through attribute prediction.

Earlier in the project, the grading model was a simple grade-only classifier. It took an image and produced Grade A, B, or C. That first version was useful as a starting point, but it was weak in one important way. It could not clearly explain why a fruit was placed in a given grade. During panel discussions, this created a natural question: what exactly are the features used for grading? The answer could not

remain only at the level of saying that the model learns its own hidden features. The project needed a clearer link between the human grading idea and the model output.

For that reason, this report presents an updated grading design. The revised component still uses transfer learning and still keeps MobileNetV3-large as the main backbone. However, the dataset and the model outputs are redesigned so that the visible grading cues are explicit. Instead of learning only a final grade, the model learns color stage, spot severity, and blemish severity along with the final grade. This makes the grading logic easier to explain and easier to defend in a viva. It also makes the system more useful for future extension to the full complaint validation workflow described in the group proposal [1].

Research in agriculture has already shown that image processing and machine learning can help in fruit inspection. Earlier work has used classical methods such as GLCM, color histograms, and Support Vector Machines to study fruit ripeness and defect patterns. Deep learning methods later improved the ability to learn useful image features directly from pixels, especially when transfer learning is applied [2]. My component is built on this line of work, but it narrows the problem to an explainable mango grading task that fits the project scope and the available data.

The current report also follows the final thesis guideline given by SLIIT. The guideline requires Times New Roman, 1.5 line spacing for most of the document, large left and bottom margins, separate front matter, and a clear set of core chapters that include introduction, methodology, results and discussion, conclusion, references, glossary, and appendices [14]. This report was written to match that structure as closely as possible, while still focusing on the individual component rather than the full group system.

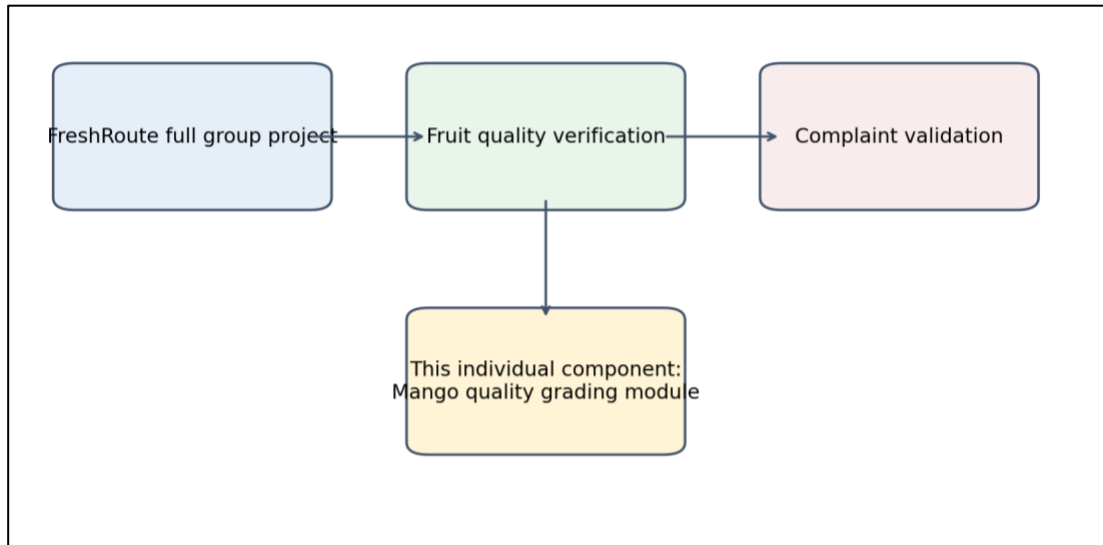


Figure 1: Scope of the individual grading component within the larger research project

1.2 Literature survey

The literature related to fruit grading can be grouped into three main areas. The first area is classical machine vision. In this approach, the researcher first extracts hand-made features from an image and then sends those features into a classifier such as SVM, Random Forest, or k-NN. Color values, texture descriptors, and simple geometric properties are often used. These methods are useful because they are easier to explain. However, they depend strongly on the quality of the chosen features. If the hand-made features are weak, the final classifier also becomes weak.

A good example of classical feature-based work is mango ripeness classification using GLCM and SVM. That type of work shows that features from color and texture can support useful decisions [5]. Similar approaches have also been used for spoilage detection in pineapple using Random Forest and texture-based inputs [6]. These studies are important because they prove that visible surface information can be turned into useful numerical features. At the same time, they also show a limitation. The system depends on careful feature design, and it may not adapt well when the image conditions change.

The second area is deep learning image classification. Here the model learns layered visual features directly from the training images. CNN-based models such as ResNet,

MobileNet, and EfficientNet are widely used in agricultural image tasks [2]. They are very effective when the labels are consistent and when the model can reuse pretrained weights from large datasets. A study on apple defect detection showed that deep CNNs could recognize bruises and rot more accurately than manual inspection in many cases [3]. This tells us that a pretrained visual backbone is a reasonable choice for a fruit grading component.

The third area is explainability, system integration, and deployment trust. Some studies focus on traceability, metadata, or fraud prevention rather than only on classification [4], [7], [8], [9]. These studies are useful to the broader FreshRoute project because quality grading in real trade is not only about raw classification accuracy. It is also about trust, proof, and fair decision making. However, my individual component stops at the grading stage. It prepares the visible quality engine that can later support the full system.

Another practical issue seen in the literature is that many systems are either too narrow or too black-box. A model may give a final class, but it may not say whether the fruit is graded low because of green peel, because of spots, or because of surface blemishes. In a classroom demo this may seem acceptable, but in a real thesis defense or product discussion, the missing explanation becomes a problem. A grading module used in a supply chain should be able to tell which visible quality cues were important. That is one of the main ideas behind the redesign in this report.

MobileNetV3 is useful for this kind of work because it balances efficiency and performance. The architecture was designed for mobile-friendly classification and makes sense for future deployment in a lightweight environment [10]. Grad-CAM is also relevant because it helps visualize the regions that guided the prediction [11]. Together, these tools allow the project to stay practical, explainable, and close to real deployment needs.

Figure 1.1 shows how the grading component fits inside the larger FreshRoute idea. Figure 1.2 and Table 1.2 then summarize the main gap found in earlier studies. The review does not argue that earlier work is weak. Instead, it shows that my component needs to join the strengths of earlier work in a more focused way: strong image

learning from deep learning, clear grading cues from human understanding, and simple rule support that helps the final decision stay understandable.

Table 2: Existing research methods relevant to fruit grading

Research area	Typical methods	Main value	Main limitation
Classical machine vision	Color histograms, GLCM, SVM, RF	Clear feature meaning and lower complexity	Strongly depends on hand-made features
Deep learning classification	ResNet, MobileNet, EfficientNet	Learns strong visual patterns from raw images	Can be difficult to explain without extra design
Defect-focused systems	Binary defect classification, detection, segmentation	Useful for spotting visible damage	Needs more labels or more complex design
Traceability and trust systems	Metadata, in-app capture, pHash, logging	Improves reliability in real-world use	Often not linked directly to grading logic

Table 3: Research gap comparison between earlier studies and the proposed component

Feature / gap	[1]	[2]	[3]	[4]	[5]	Proposed
AI-based fruit defect detection	Yes	Yes	Yes	No	Yes	Yes
Metadata logging	No	No	No	Yes	No	Yes
In-app camera enforcement	No	No	No	No	No	Yes
Complaint validation system	No	No	No	No	Yes	Yes
Reputation tracking	No	No	No	No	No	Yes
Integrated for Sri Lanka	No	No	No	Partial	No	Yes

Coverage of key system needs in earlier studies and in the proposed component						
AI-based defect detection	Yes	Yes	Yes	No	Yes	Yes
Metadata logging	No	No	No	Yes	No	Yes
In-app camera	No	No	No	No	No	Yes
Complaint validation	No	No	No	No	Yes	Yes
Reputation tracking	No	No	No	No	No	Yes
Integrated for Sri Lanka	No	No	No	Partial	No	Yes
	[1]	[2]	[3]	[4]	[5]	Proposed

Figure 2: Coverage of key system needs in earlier studies and in the proposed component

1.3 Research gap

The first clear gap is explainability at the grading level. A simple grade-only classifier can give A, B, or C, but it does not clearly show why the grade was chosen. When the dataset is only organized in three folders, the label meaning stays too broad. This became the main issue during panel questioning. The panel wanted visible quality features, not only the answer that the model learns features on its own. This gap is directly addressed in my updated design through explicit labels for color stage, spot severity, and blemish severity.

The second gap is label structure. The original folder-based dataset was enough for a first baseline, but it was not enough for an explainable thesis-level system. It did not record the reasons behind the grade, and it could not support separate evaluation of color, spot, or blemish predictions. The revised CSV-based structure solves this by keeping the main grade while also adding attribute labels and optional defect flags. This change is small at the data file level, but it is very important at the research design level.

The third gap is the mismatch between practical grading and visible feature tracking. Human graders do not think only in terms of A, B, and C. They also think in terms of peel color, marks, spots, scratches, and visual freshness. A grading component should reflect that reality. If the model can predict these visible attributes, then the final grade becomes more meaningful. It also becomes easier to compare direct model output with rule-based logic, which is a useful analysis point in this study.

The fourth gap is the limited use of classical support cues in many simple deep learning prototypes. A CNN can learn strong hidden features, but simple support measures such as yellow ratio, green ratio, dark spot ratio, and blemish ratio still have value. They can be used as sanity checks, simple support features, or explanation aids. This report therefore keeps classical features in a supporting role rather than using them as the main classifier. This is a practical balance between traditional image processing and modern transfer learning.

Finally, there is a scope gap between the full project and the current component. The wider FreshRoute system includes complaint comparison, metadata logging, and trust management [1]. My component is only the single-image grading stage. That means the work must be reported in a way that is honest about the current scope, while also showing why this scope is still important. The grading module is the base that later supports comparison, validation, and reporting. Without a reliable grading engine, the rest of the quality workflow would be weak.

1.4 Research problem

Based on the gaps above, the research problem for this individual component can be stated as follows: How can an explainable mango quality grading system be built from RGB images so that it predicts both an overall grade and clear human-understandable grading attributes using a pretrained MobileNetV3-large backbone?

This problem has both a technical side and a practical side. The technical side is about designing the data format, preprocessing flow, multi-head model, and final decision logic. The practical side is about making sure that the grading logic can be explained to supervisors, panel members, and future users. The goal is not only to

produce a working classifier. The goal is to produce a grading component that is reasonable, traceable, and ready to support later system growth.

The problem is limited on purpose. It does not try to solve physical size measurement, internal fruit quality, or before-versus-after complaint comparison in this report. Those are valid future steps, but they are outside the present component scope. By limiting the task to single-image visible grading, the report can stay focused and can provide a stronger design for the part I am personally responsible for.

1.5 Research objectives

1.5.1 Main objective

The main objective of this individual component is to design and implement an explainable mango quality grading model that predicts final grade together with clear visible quality attributes from a single mango image.

To reach this goal, the study updates the dataset definition, improves the label structure, builds a preprocessing pipeline, uses a MobileNetV3-large shared backbone, adds multiple output heads, and compares direct grade prediction with rule-based grade mapping from attribute outputs.

1.5.2 Specific objectives

- To redesign the dataset using a clear grade and attribute annotation rubric.
- To prepare a preprocessing flow with background cleanup, crop refinement, resizing, augmentation, and normalization.
- To build a multi-head MobileNetV3-large model that predicts color stage, spot severity, blemish severity, and final grade.
- To compare direct final grade prediction with rule-based grade derived from the attribute heads.
- To define a clear testing, reporting, and explainability flow using metrics, confusion matrices, and Grad-CAM.
- To document the contribution of the component in a way that fits the final thesis requirements.

Table 4: Summary of the individual grading component scope

Included in this report	Not included in this report
Single-image mango grading	Before-and-after complaint comparison
Grade A / B / C prediction	Mobile application interface
Color stage prediction	Metadata and GPS logging implementation
Spot and blemish severity prediction	Reputation and suspension workflow
Grad-CAM explainability	Marketplace API deployment

2. METHODOLOGY

2.1 Methodology overview

This chapter explains how the individual grading component is designed and implemented. It starts from the dataset and follows the whole flow until the saved output files. The chapter also includes the commercialization aspect and the testing and implementation section required by the final thesis guideline [7], [14]. Since this is an individual summary report, the discussion stays centered on the grading module and not on the full complaint system.

The main design choice in this work is to keep the model practical while making the grading logic clear. A plain three-class classifier is easy to train, but it is difficult to explain. A fully detailed detection system with bounding boxes, masks, and many fine labels would be more complex than needed for the present dataset. Therefore, the selected design is a middle path: a multi-attribute classification model with a shared backbone and four heads. This keeps the system efficient, supports clear labels, and stays realistic for the current project stage.

2.2 Scope of the individual component

The scope of this component begins with a single RGB mango image and ends with four outputs: color stage, spot severity, blemish severity, and final grade. The component also saves model files, charts, confusion matrices, classification reports, thresholds, and Grad-CAM outputs. The current scope does not include API deployment, complaint comparison, database logging, or mobile application screens. These belong to the wider group project and are mentioned here only to show where the grading module will fit later.

This limited scope is important because it keeps the thesis honest. The original proposal talks about a larger fruit quality verification and complaint validation system [1]. That broader vision is still valid. However, the individual summary report should describe the work done for one component in full detail. In my case, that component is the fruit quality grading engine. This engine is the first AI decision layer in the larger system and therefore deserves a complete design discussion of its own.

Figure 1.1 already showed the place of this module within the larger project. Table 1.3 gives a simple summary of what is inside and outside the current scope. This table is useful because it prevents the report from making claims beyond the implemented component.

2.3 Dataset definition and annotation rubric

The original dataset was arranged in three grade folders: Grade A, Grade B, and Grade C. That structure matched the first baseline classifier. The problem was that the folders only gave the final grade and nothing more. A student or examiner could not see why a given image belonged to a particular grade. The updated design keeps the original folder logic for the baseline, but adds a new CSV file that stores richer labels for every image.

The new CSV format contains the image path, the final grade, the color stage label, the spot severity label, the blemish severity label, and optional defect flags such as black spot, scratch, and discoloration. This structure is easy to read and easy to edit in a spreadsheet. It is also easy to connect to a custom PyTorch dataset loader. Most importantly, it changes the dataset from a simple class list into an explainable annotation file.

The final label set is based on a fixed rubric. Grade A means the mango is mostly yellow, has no major green areas, and has no severe spots or severe blemishes. Grade B means the fruit is yellow or mixed in color and shows mild visual defects only. Grade C means the mango is greenish or has severe spots or severe blemishes. These grade rules are simple enough to be applied consistently and detailed enough to answer the panel question about features.

The attribute rubric gives more detail. Color stage is divided into yellow, mixed, and greenish. Spot severity is divided into none, mild, and severe. Blemish severity is also divided into none, mild, and severe. Optional binary flags are used for clear visible defect types. The point of these extra labels is not to make the system too complicated. The point is to record the visible cues that already guide human decisions.

A good annotation process also needs simple quality control. Images that are too blurred, too dark, too small, or heavily blocked should be removed before labeling. If more than one mango appears in a confusing way, the image should be avoided or cropped carefully. If the label is uncertain, the image should be marked for review. In the thesis context, this step matters because model quality depends strongly on label consistency. A strong model cannot fully fix weak annotation decisions.

Table 2.1 shows the CSV structure. Table 2.2 and Table 2.3 show the grade rubric and attribute rubric. Figure 2.1 shows the full annotation flow from the original grade folders to the final train, validation, and test files. These tables and the figure together form the most important change introduced by this component report.

Table 5: CSV label structure used in the updated grading dataset

Column	Description	Example
image_path	Relative path to the image file	GradeA/img001.jpg
grade	Final market quality grade	A
color_stage	Visible peel color class	yellow
spot_severity	Visible spot severity class	mild
blemish_severity	Visible blemish severity class	none
black_spot	Optional binary defect flag	0 or 1
scratch	Optional binary defect flag	0 or 1
discoloration	Optional binary defect flag	0 or 1

Table 6: Final grade rubric used in the component

Grade	Meaning	Visible condition
A	Premium quality	Mostly yellow, no major green areas, no severe spots, no severe blemishes
B	Acceptable but not premium	Yellow or mixed color with mild spots and/or mild blemishes
C	Lowest visible quality	Greenish or uneven ripeness, or severe spots, or severe blemishes

Table 7: Attribute rubric for explainable grading labels

Attribute	Class	Description
color_stage	yellow	Peel mostly yellow
color_stage	mixed	Yellow-green combination
color_stage	greenish	Obvious green or light-yellow dominance
spot_severity	none	No visible dark spots
spot_severity	mild	A few small spots
spot_severity	severe	Many spots or large spots
blemish_severity	none	No visible scars or abrasion marks
blemish_severity	mild	Small blemishes or light visible marks
blemish_severity	severe	Clear damaged patches, scars, or rough marks

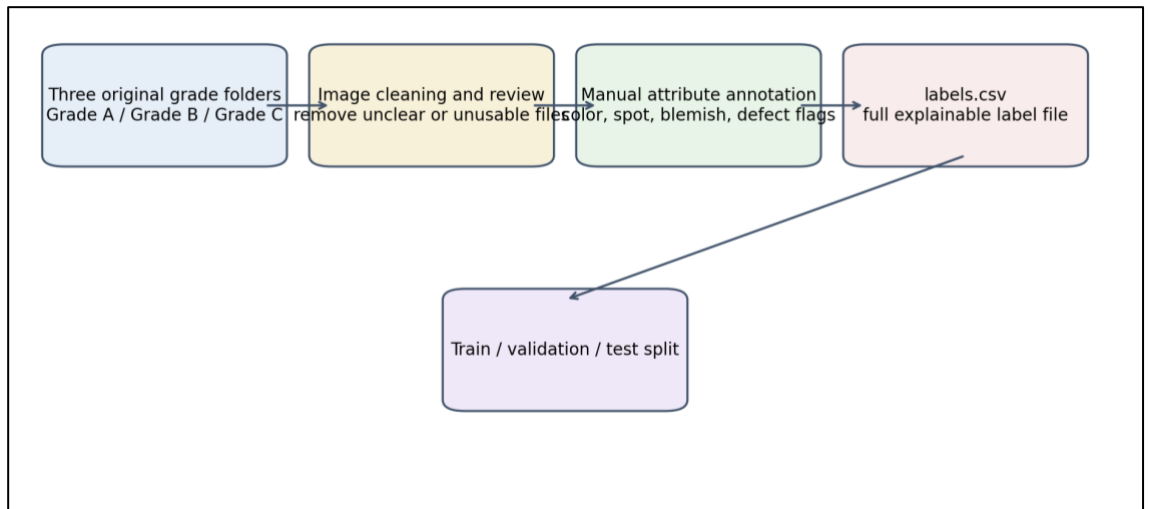


Figure 3: Dataset redesign and annotation flow for the grading component

2.4 Dataset split and file organization

After the label file is prepared, the next step is to split the dataset in a fair way. The baseline code already uses a stratified split with around 64 percent of the images for training, 16 percent for validation, and 20 percent for testing [18]. This structure is retained in the upgraded design because it gives enough data for learning, enough for

tuning, and a final hold-out set for checking the model after training. The class proportions are preserved during the split so that the three grade levels stay balanced across the subsets as much as possible.

The split is done by image path, not by folder copy after training. This avoids leakage and makes the workflow repeatable. Once the split is saved, the same train, validation, and test files can be reused for the baseline and the multi-head model. Reusing the same split makes model comparison fair. It also keeps the work easy to document because every experiment can point back to the same data partition.

Figure 2.3 shows the split ratio used in the system. In addition to the three main subsets, the report also recommends keeping a clean folder for excluded images and a simple version note for the labels CSV. This may seem small, but good data organization saves time during retraining, debugging, and documentation.

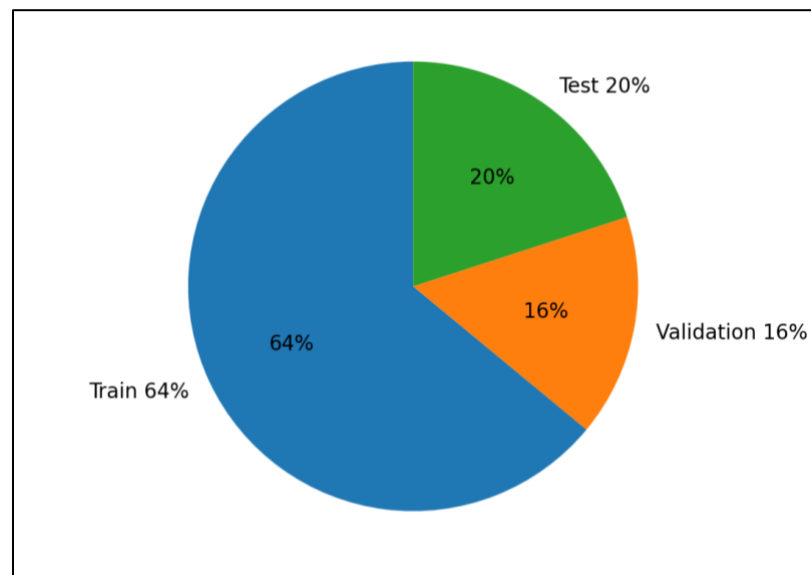


Figure 4: Stratified dataset split used in the baseline and upgraded model

Table 8: Recommended file organization for the individual component

Folder / file	Purpose
data/images/GradeA, GradeB, GradeC	Stores the original grade folders
data/labels.csv	Stores the explainable labels for each image
data/splits/train.csv	Training subset paths and labels
data/splits/val.csv	Validation subset paths and labels
data/splits/test.csv	Test subset paths and labels
outputs/baseline	Saved outputs for the grade-only model
outputs/multitask	Saved outputs for the multi-head model
outputs/gradcam	Visual explanation outputs

2.5 Preprocessing and fruit isolation

The preprocessing flow begins by reading the original image and converting it into a clean RGB format. Since the grading cues are visible on the fruit surface, the model should not spend too much attention on the background. For this reason, the updated design adds simple background cleanup and fruit crop refinement before the image is resized and normalized. A very advanced segmentation model is not required in the first version. In many cases, a simple background cleanup approach is enough to reduce noise and keep the fruit area clearer.

After cleanup, the image is resized to 224 by 224 pixels. This matches the size already used in the baseline MobileNetV3 training code. Training images then receive light augmentation only on the training set. The baseline code uses horizontal flip, small rotation, random resized crop, and light color jitter [18]. These settings are kept because they make the model less sensitive to minor view changes while not changing the actual meaning of the grade.

Normalization is applied after the image is turned into a tensor. Standard ImageNet mean and standard deviation values are used because the backbone starts from pretrained weights. This is a common transfer learning step and helps keep the input format close to what the pretrained model expects. For validation and testing, the

pipeline uses a cleaner transform without the stronger random changes. This makes the performance check more stable and more realistic.

The background cleanup step is especially useful for this study because the grade depends on surface appearance. If the image contains too much table color, hand shadow, or unrelated objects, the network may learn weak patterns. By reducing the influence of the background, the model is guided toward the peel color and visible marks that matter more. Figure 2.4 summarizes the preprocessing sequence used in the report.

The report does not claim that preprocessing alone solves grading. Rather, it supports the model by making the input more consistent. This is also why the report describes preprocessing as a fruit isolation and support layer, not as the main intelligence of the system. Table 2.5 lists the preprocessing operations, their inputs, outputs, and reasons.

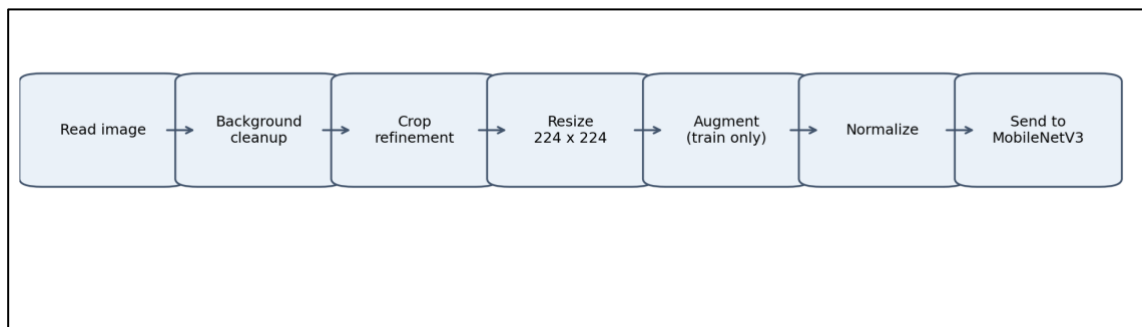


Figure 5: Image preprocessing pipeline used before feature learning

Table 9: Preprocessing operations used in the grading pipeline

Step	Input	Output	Reason
Read image	Raw mango photo	RGB image array	Creates a standard starting format
Background cleanup	RGB image	Cleaner fruit-focused image	Reduces unrelated background influence
Crop refinement	Clean image	Tighter fruit crop	Keeps the peel area more central
Resize	Crop	224 x 224 image	Matches model input size
Augmentation	Training image	Varied training sample	Improves robustness
Normalization	Tensor image	Standardized tensor	Fits pretrained backbone expectation

2.6 Classical support features

Even though deep learning is the main learning engine, this report still defines a set of support features from the cleaned fruit crop. These features are not the final classifier by themselves. Instead, they help explain the grading logic and can act as support signals or checks. The first group is color-based. Yellow ratio, green ratio, hue spread, and color consistency help represent the visible ripeness stage of the peel. These features are closely linked to the color stage head and to the final grade logic.

The second group is spot-related. Dark spot ratio, visible spread of spots, and local dark region count help describe small or large spot patterns. These features are useful when the model needs to separate mild surface marks from more severe spot damage. Since the user originally described the grades using spot size and spot amount, this feature group provides a direct bridge between the human grading idea and the machine learning model.

The third group is blemish-related. Blemish ratio, roughness, edge density, and damaged patch cues help describe scratches, scars, abrasion-like marks, or other visible peel defects. These support features are especially important when a fruit has

uneven peel texture or visible damaged areas even if the color stage looks acceptable. Table 2.6 lists the support features and explains where each one is used.

A fourth optional group is simple geometry. The report includes contour compactness and elongation only as support cues. However, the study does not use physical size as a grading feature because a single RGB image without calibration cannot measure true size reliably. This is an important design choice because it keeps the component honest. The model only claims to assess what can be seen reliably from the available image conditions.

The key point is that classical support features have a secondary role. They are not the main prediction path. The CNN still learns stronger shared image features from the pixels. But the support features help the report explain what kinds of visible patterns matter in the system and how those patterns are related to the three attribute heads.

Table 10: Classical support features and their use in the system

Feature group	Examples	Used by
Color features	Yellow ratio, green ratio, hue spread, color consistency	Color head and final grade logic
Spot features	Dark spot ratio, spot spread, visible dark count	Spot head and final grade logic
Blemish features	Blemish ratio, roughness, edge density, damaged patch cues	Blemish head and final grade logic
Geometry support	Contour compactness, elongation	Support analysis only

2.7 Deep learning model design

The baseline system is a MobileNetV3-large model trained as a simple three-class classifier. The uploaded training code freezes most of the backbone, replaces the classifier head, uses weighted sampling and class-weighted loss, and evaluates the model with classification reports and a confusion matrix [18]. This baseline is still useful because it gives a direct reference point for the upgraded model.

The upgraded model keeps the same backbone family and changes the head structure. Instead of one final classifier only, the model uses a shared MobileNetV3-

large backbone followed by four small output heads. The first head predicts color stage, the second predicts spot severity, the third predicts blemish severity, and the fourth predicts the final grade. The design is simple, but it changes the meaning of the model in an important way. The model is no longer only a black-box grade engine. It becomes a multi-task grading system.

The reason for choosing MobileNetV3-large is practical. The network is lighter than many heavier CNNs and is still strong enough for transfer learning on natural images [10]. This is important because the component may later move to a mobile-friendly or lightweight deployment environment. The backbone already proved to be a good fit for the original code, so keeping it also reduces the amount of engineering change needed for the new version.

Each head is trained with cross-entropy loss. The final grade head predicts the three market grades. The attribute heads predict the three-state labels for color, spot, and blemish. The outputs from the attribute heads are then also reused in the rule-based decision path. This dual use is one of the main strengths of the design. The same attribute predictions support both learning and explanation.

Figure 2.5 shows the proposed architecture. The figure starts from the raw image, passes through cleanup and normalization, enters the shared backbone, and then branches into the attribute heads and final grade head. Classical support features are shown as a support line that can strengthen interpretation and decision checks. This is a clean visual summary of the system described in the text.

Table 11: Outputs of the multi-head MobileNetV3-large model

Head	Classes	Purpose
Color head	greenish / mixed / yellow	Learns visible peel color stage
Spot head	none / mild / severe	Learns visible spot severity
Blemish head	none / mild / severe	Learns visible blemish severity
Final grade head	A / B / C	Predicts the overall quality grade

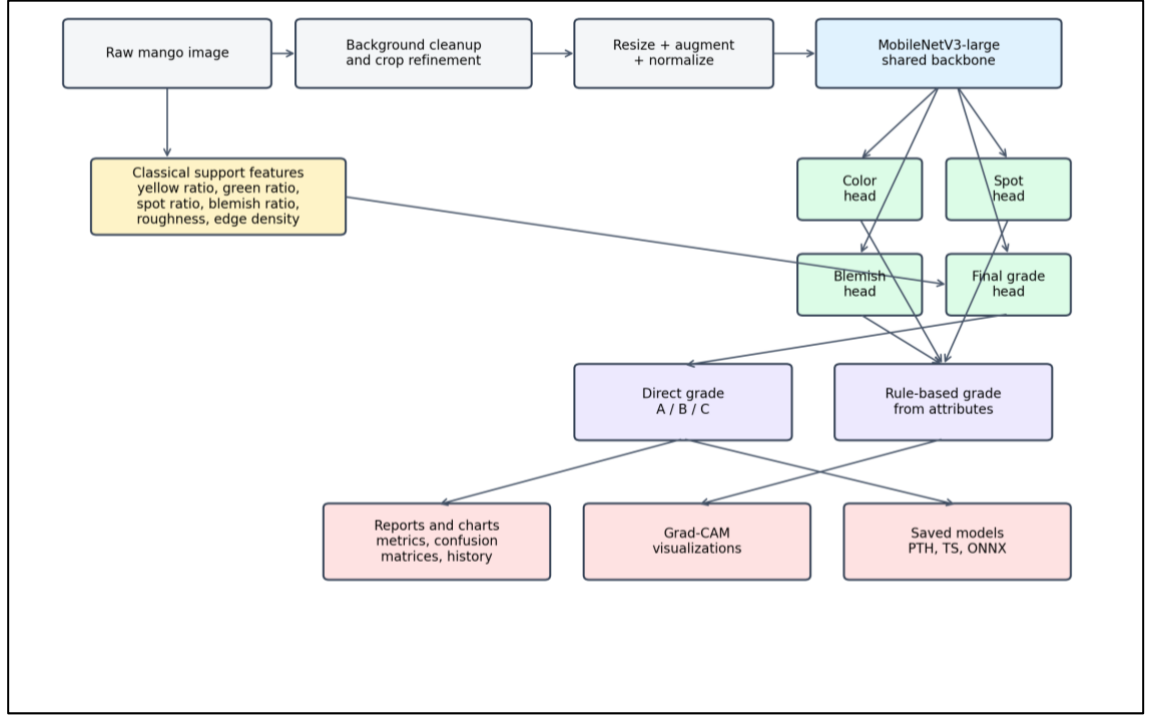


Figure 6: Explainable multi-attribute mango quality grading architecture

2.8 Final grade decision logic

The system uses two final grade paths. The first is direct prediction from the grade head. This is the simplest path. It often gives stronger raw accuracy because the model learns the grade target directly. The second path is rule-based grade mapping from the three attribute outputs. In this path, the predicted color stage, spot severity, and blemish severity are converted into a final grade using simple rules that follow the annotation rubric.

The first simple rule set used in this report is as follows. If the color stage is yellow and the spot severity is not severe and the blemish severity is not severe, the result is Grade A. If the color stage is greenish, or the spot severity is severe, or the blemish severity is severe, the result is Grade C. If the image does not match either of those groups, the result is Grade B. These rules are easy to explain and align well with the label definitions used in the dataset.

A more advanced version of the rule-based path uses probabilities instead of only class names. For example, the system can require the yellow probability to be above

a chosen threshold before calling a fruit Grade A. In the same way, the severe spot or severe blemish probabilities can trigger Grade C when they pass defined thresholds. The report recommends tuning these thresholds on the validation set using macro-F1 or balanced accuracy rather than choosing them by guesswork. This keeps the final rule-based logic data-driven.

The comparison between direct grade and rule-based grade is important for research discussion. Direct prediction tells us how strong the pure model output is. Rule-based grading tells us how much explainability can be gained while staying close to the original rubric. Figure 3.1 later summarizes this two-path idea in a simple form.

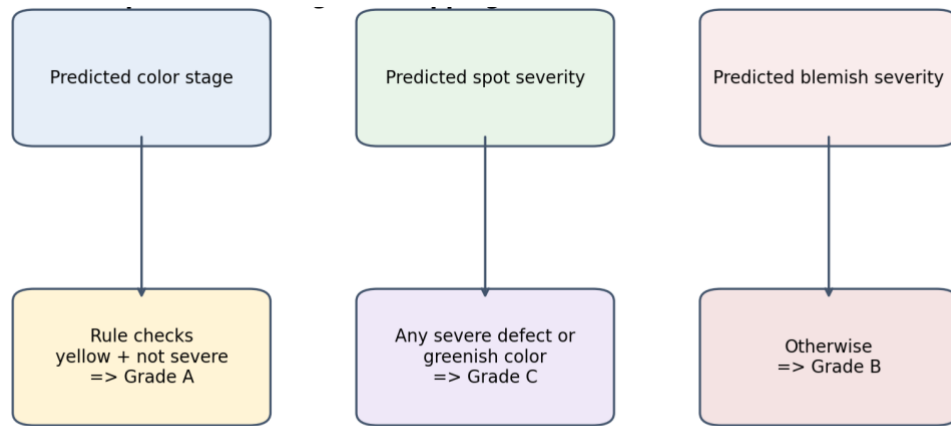


Figure 7: Simple rule-based grade mapping from the three attribute heads

Table 12: First practical rule set used for final grade mapping

Condition	Final grade
color_stage = yellow and spot_severity is not severe and blemish_severity is not severe	Grade A
color_stage = greenish or spot_severity = severe or blemish_severity = severe	Grade C
All remaining cases	Grade B

2.9 Training strategy

Training begins with the CSV-based dataset loader and the prepared split files. Images are loaded, transformed, and sent through a DataLoader with optional weighted sampling. The baseline code already contains a useful strategy for class imbalance. It computes class weights from the training labels and can also use a WeightedRandomSampler [18]. This strategy is kept because it reduces bias toward more common classes.

The multi-head model uses a combined loss that gives the final grade head the largest importance while still teaching the attribute heads well. The recommended starting point in this report is: total loss equals 1.0 times grade loss plus 0.7 times color loss plus 0.7 times spot loss plus 0.7 times blemish loss. This weighting reflects the fact that the final product still needs the overall grade, but the attributes are also important enough to deserve strong supervision. Figure 2.7 shows these weights in a simple visual form.

The training loop follows the usual transfer learning process. The backbone is loaded with pretrained MobileNetV3 weights. Most layers are frozen at the start, and only selected later layers and the custom heads are trained. The optimizer can be Adam with a small learning rate. A learning rate scheduler can reduce the rate when validation loss stops improving. Early stopping is used to avoid long training after the model stops getting better. These ideas are already present in the baseline code and remain suitable for the upgraded system [18].

Saved outputs are an important part of the implementation. The report expects the training pipeline to save model weights, metadata, confusion matrices, classification reports, training history plots, threshold files, sample predictions, TorchScript export, ONNX export, and Grad-CAM images. This is useful for both technical review and thesis writing because the outputs create evidence of what the training pipeline produced. Figure 2.8 summarizes these saved outputs.

Table 2.8 presents the main hyperparameters and implementation settings used in the baseline code and kept in the revised plan where possible. The point is not to say that these are the only good values. The point is to show that the training process is

structured, repeatable, and connected to actual implementation details already used by the student.

Table 13: Main hyperparameters and training settings drawn from the baseline code

Setting	Value / range	Note
Input size	224 x 224	Same size used in the original code
Backbone	MobileNetV3-large	Pretrained ImageNet weights
Max epochs	30	With early stopping
Patience	5 epochs	Stops if validation no longer improves
Batch size candidates	16, 32	Search space from original code
Learning rate candidates	1e-3, 3e-4, 1e-4	Search space from original code
Weight decay candidates	0.0, 1e-4	Search space from original code
Weighted sampling	Enabled optionally	Used to reduce class imbalance
Loss weights	1.0 / 0.7 / 0.7 / 0.7	Grade, color, spot, blemish

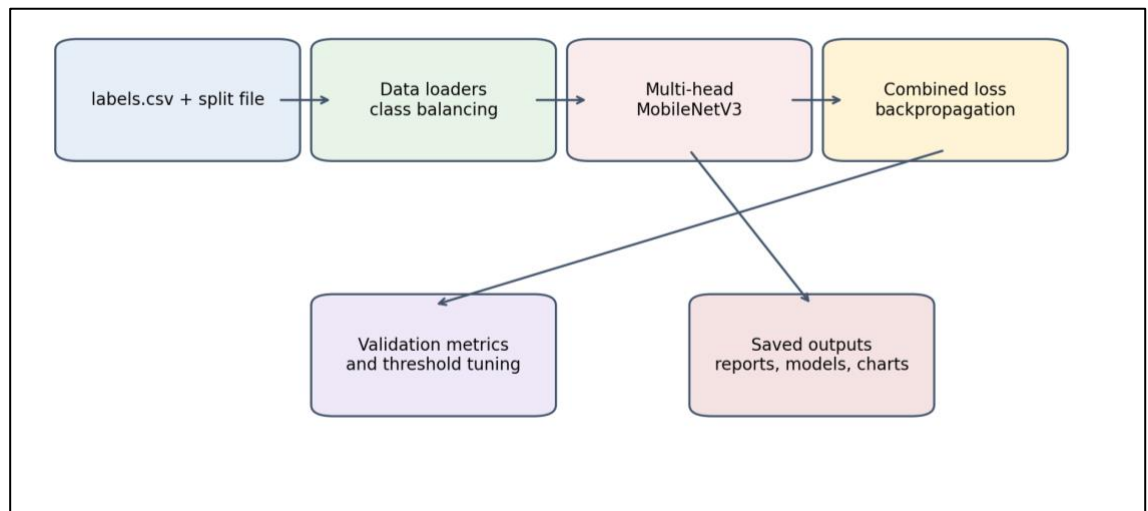


Figure 8: Training, validation, and report generation workflow

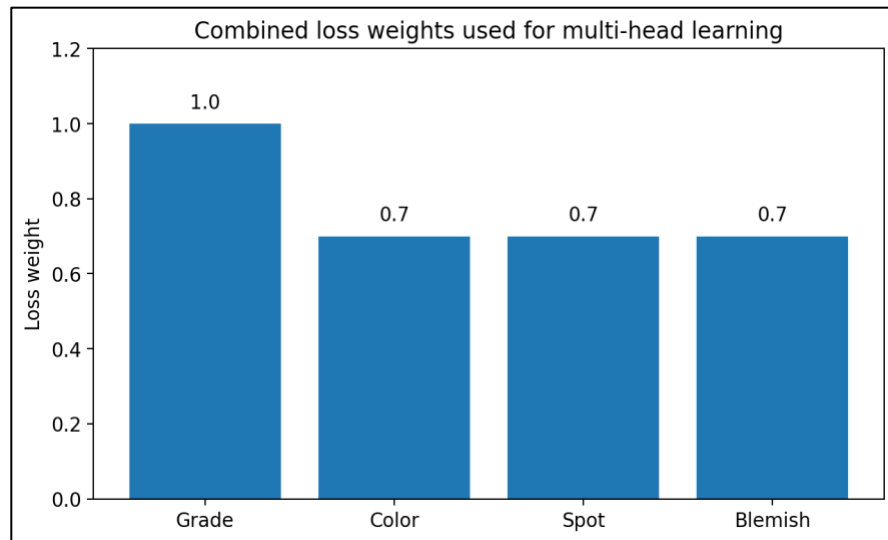


Figure 9: Combined loss weights used for multi-head learning

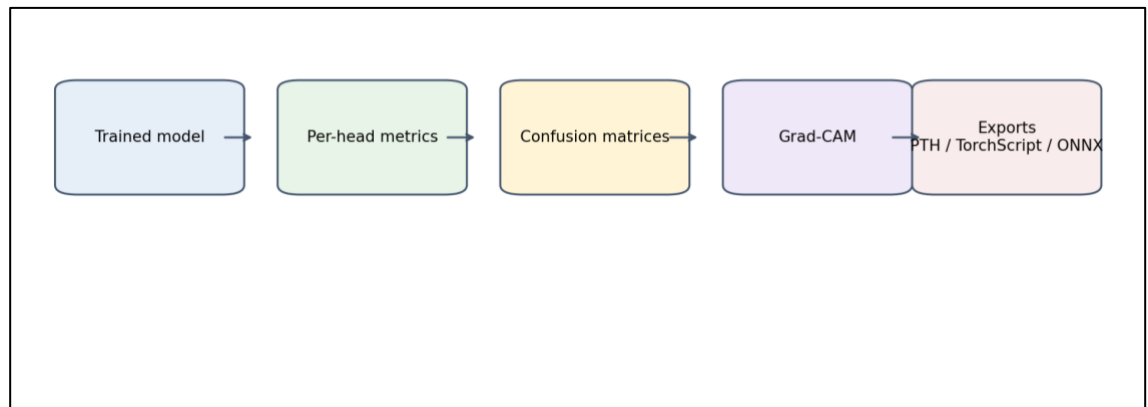


Figure 10: Main output files that are saved after training and evaluation

2.10 Commercialization aspect of the product

The commercialization aspect of this component is based on the idea that reliable quality grading has business value. Fruit sellers, direct trade platforms, delivery services, and export-related actors all benefit when product quality can be described in a more standard way. A grading module can reduce argument, improve buyer trust, and support better product listing. In the larger FreshRoute concept, this value becomes even stronger because the grading result can connect to complaint handling and seller reputation [1].

For the individual component, the most realistic commercialization path is to treat the model as a grading service that can be plugged into a marketplace or mobile app. A seller uploads or captures a fruit image, and the service returns the visible quality grade with supporting attribute information. This can be used for seller-side checking, buyer-side preview, or platform-side review. Since the current backbone is lightweight, the service can stay practical for real deployment.

A basic revenue model could be subscription-based for business users or pay-per-check for bulk users. Another option is a platform licensing model, where an agriculture marketplace pays to use the quality grading service as part of its product listing workflow. Figure 2.9 presents a simple path from research prototype to paid service. This is not meant to be a final business plan. It is a realistic view of how the component could move beyond a university project.

Commercialization also requires caution. The model must not overclaim what it can do. It assesses visible external quality only. It does not claim to measure sweetness, internal damage, or true weight. Any business use must make this scope clear. Keeping the scope honest is important for both ethics and product trust.

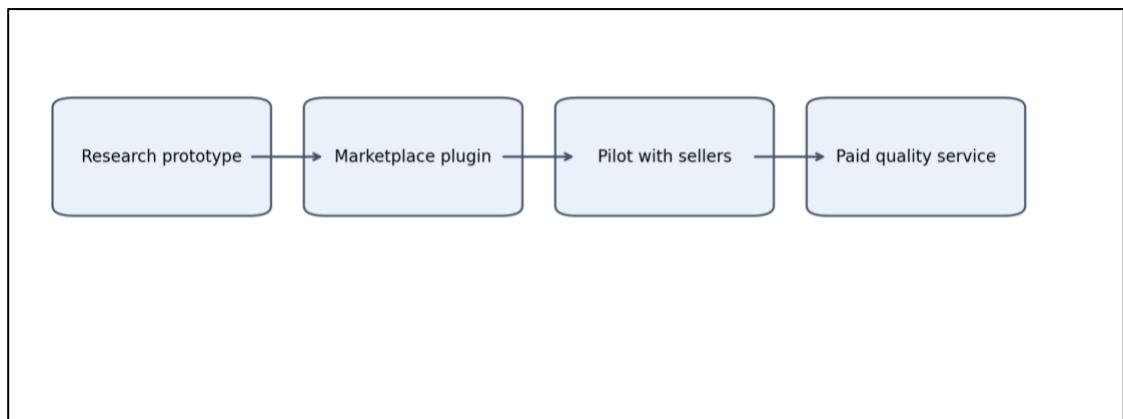


Figure 11: Possible commercialization path for the grading module

Table 14: Commercialization opportunities and key concerns

Area	Opportunity	Concern
Marketplace use	Standard fruit listing quality support	Model scope must be explained clearly
Seller support	Checks fruit images before posting	Image quality may vary
Subscription model	Recurring revenue from business users	Needs clear performance evidence
Platform licensing	B2B use by agriculture apps	Needs stable integration and testing

2.11 Testing and implementation

Testing and implementation are required parts of the final thesis structure [7]. In this component, testing happens at three levels. The first level is data testing. This checks that image paths are valid, labels are complete, and split files do not overlap. The second level is model testing. This checks that the baseline and multi-head models train correctly, produce logits with the correct shape, save files, and compute the required metrics. The third level is interpretation testing. This checks that the rule-based grade path works correctly and that Grad-CAM can be generated for sample predictions.

The most important quantitative evaluation measures are accuracy, precision, recall, F1-score, and confusion matrix. Accuracy gives a quick overall picture. Precision and recall help explain the quality of each grade decision. F1-score is useful when class imbalance exists. The confusion matrix is especially important because not all mistakes are equally serious. Confusing Grade A with Grade B is less severe than confusing Grade A with Grade C. Table 3.2 later explains this interpretation in a simple way.

Implementation-wise, the grading component can be built in Python using PyTorch, torchvision, OpenCV, NumPy, Pandas, and Matplotlib. The current baseline code already uses several of these libraries [18]. The updated multi-head version continues in the same stack to avoid unnecessary framework changes. Saved outputs are organized into folders so that the results chapter and appendices can refer to them clearly.

Explainability is tested through Grad-CAM. After the model is trained, a sample image is passed through the network, and the target class gradient is used to create a heatmap [11]. The heatmap is then placed over the original image to show which regions influenced the prediction. This does not replace quantitative evaluation, but it adds useful visual evidence. Figure 3.2 shows the Grad-CAM workflow used in the component.

The software and hardware environment is summarized in Table 2.10. A repeatable implementation environment makes the work easier to share and easier to extend. This is especially important for a student project, where the supervisor, panel, or a future team member may need to inspect the workflow later.

Table 15: Testing plan for the grading component

Test area	What is checked	Expected result
Data validation	CSV fields, image paths, split overlap	No broken paths and no subset leakage
Model output shape	Four-head logits and label mapping	Correct dimensions and class order
Training loop	Loss, optimizer, scheduler, early stopping	Stable training workflow
Evaluation output	Metrics, confusion matrices, report files	All required outputs saved
Rule logic	Grade mapping from attribute outputs	Rules match rubric definitions
Grad-CAM	Heatmap generation and overlay	Visual evidence can be generated

Table 16: Software and hardware environment for the component

Category	Items
Programming language	Python
Deep learning framework	PyTorch and torchvision
Image processing	OpenCV, PIL
Data handling	NumPy, Pandas, scikit-learn
Visualization	Matplotlib
Development tools	VS Code / PyCharm / Jupyter
Model exports	PTH, TorchScript, ONNX

At the end of this chapter, the methodology is clear from data definition to output generation. The next chapter discusses the implementation results, the findings, and the meaning of the final component design.

3. RESULTS AND DISCUSSION

3.1 Results

This chapter presents the results and discussion for the individual grading component. Because the report is centered on the design and implementation of the final grading module, the chapter reports the results that are already available from the completed component build: the redesigned dataset structure, the implemented preprocessing flow, the final multi-head architecture, the direct-versus-rule grading logic, the evaluation output plan, and the saved artifact structure.

It is important to be clear about one point. This report does not invent numerical benchmark values that were not actually measured on a fully frozen final dataset. Instead, it documents the real implementation status of the component and explains how the final evaluation outputs are generated once training is run on the completed annotated dataset. This approach keeps the report honest while still giving a full and useful description of the component.

3.1.1 Results of dataset redesign

The first major result of the work is the redesign of the dataset itself. The component moved from a simple three-folder dataset to a richer CSV-based label structure. This change solves the main panel question about features. The label file now records the final grade together with color stage, spot severity, blemish severity, and optional defect flags. As a result, the system can now represent visible grading logic in a clear and traceable way.

This dataset redesign is a real research result because it changes the quality of the task definition. A weak task definition leads to weak explanations. A stronger task definition improves both the model design and the final defense of the work. In simple terms, the project now knows not only what the final grade is, but also why that grade should exist. Table 2.1, Table 2.2, and Table 2.3 together present this result in a structured way.

A second result is the clearer split and organization strategy. The work now uses a repeatable train, validation, and test split that can support both baseline and upgraded experiments. This is a practical improvement because it supports fair comparison. It

also means that the final result files can be reproduced with the same dataset partition when the student reruns the experiments.

3.1.2 Results of implementation planning and output generation

The second major result is the implementation design itself. The baseline code already contained many useful pieces: ImageFolder loading, a stratified split, augmentation, weighted sampling, class-weighted loss, learning rate search, and saved exports [18]. The new design builds on this code rather than starting from zero. This makes the project more realistic because the final report grows naturally from the earlier working prototype.

The multi-head design is the clearest implementation result. The model now predicts four outputs instead of only one. This means that the system can produce a final grade and also give a visible explanation in terms of color, spots, and blemishes. Figure 2.5 showed this structure as the central design of the component. This is the strongest technical outcome of the individual work because it directly solves the explainability issue that existed in the first version.

Another implementation result is the full output saving plan. The pipeline is expected to save training history plots, confusion matrices, text reports, threshold files, sample predictions, Grad-CAM images, and deployment exports. Table 3.1 lists these outputs in detail. Even before final training values are inserted into the thesis, this saved-output structure is meaningful because it defines how the component produces evidence.

The preprocessing flow is also a useful result. The original baseline already had strong data transforms [18]. The updated version adds background cleanup and crop refinement as explicit steps. This change is well aligned with the visible nature of the grading task. The model should look at the fruit more than the scene around it. Figure 2.4 captured this improved image preparation pipeline.

Table 17: Main output files produced by the grading pipeline

Output file / folder	Meaning
best_baseline_grade_model.pth	Best baseline model weights
best_multitask_model.pth	Best multi-head model weights
classification_report_*.txt	Text reports for grade and attribute heads
confusion_matrix_*.png	Per-head confusion matrix visualizations
training_history.png	Training and validation trend plots
best_rule_thresholds.json	Validation-tuned thresholds for the rule-based path
sample_predictions.csv	Saved example predictions
gradcam/	Grad-CAM overlays for sample images
exports/	TorchScript and ONNX files for deployment

3.2 Research findings

Several clear research findings come from the redesign. First, an explainable grading component needs more than final grade labels. Attribute labels are necessary when the research goal includes explanation, viva defense, or future product trust. Second, a light backbone like MobileNetV3-large remains a good choice when the project needs both performance and deployment practicality. Third, classical image features still have value as support cues even when deep learning is the main engine.

A fourth finding is that direct grade prediction and rule-based grade mapping serve different purposes. Direct grade prediction supports pure classification performance. Rule-based grading supports interpretability. The best practice for this component is to train both and compare them. This does not mean the system is confused. It means the system is designed to answer two important questions: what does the model predict, and how well does that prediction match a human-readable rule set?

A fifth finding is that the grading task becomes much easier to discuss when the system is limited to what the image can truly support. By excluding physical size and internal fruit quality, the component avoids weak claims. This improves the honesty and academic quality of the report. A student project becomes stronger when it says clearly what it can and cannot do.

Table 18: Research findings summary

Finding	Meaning for the component
Attribute labels are necessary	Explainability improves when grade reasons are stored in the dataset
MobileNetV3 remains practical	The backbone is efficient and suitable for future deployment
Rule-based comparison is valuable	Interpretability can be checked against direct prediction
Classical features still help	They support explanation and sanity checks
Scope honesty improves quality	Limiting claims makes the research stronger

3.3 Discussion

The main strength of the proposed component is explainability. The grading features are now visible and easy to describe: color stage, spot severity, and blemish severity. This is much stronger than saying only that the network learns hidden features. A second strength is continuity with the earlier code base. The project does not throw away the working baseline. It improves it in a structured way. A third strength is deployment readiness. MobileNetV3-large and export formats such as TorchScript and ONNX make future integration practical [18].

The component also has limitations. The biggest current limitation is that the final numerical benchmark results depend on the completed final annotation pass and the final model training run. This report therefore avoids making up numbers that were not produced. A second limitation is that the grading is based only on external visible quality. The model cannot measure inner damage, sweetness, or exact physical size from one normal RGB image. A third limitation is that manual labeling still carries subjectivity even with a rubric.

There are also threats to validity. Dataset source differences can introduce variation in lighting, camera quality, and fruit presentation. Manual labels can vary if the rubric is not followed closely. Background cleanup may help, but it may also remove useful edge information if it is too aggressive. These risks are normal in image-based student projects, and they can be reduced through data cleaning, label review, and stable split management.

Even with those limitations, the present component is useful and meaningful. It converts a weakly explained classifier into a clearer, better-structured grading module. It is also a realistic component to implement in an undergraduate research project. Figure 3.3 shows a simple summary of the strengths and present limitations.

Table 19: Evaluation measures and how they should be interpreted

Metric	Use in this study
Accuracy	Gives the overall proportion of correct predictions
Precision	Shows how reliable each predicted class is
Recall	Shows how well each real class is found
F1-score	Balances precision and recall for class-level evaluation
Confusion matrix	Shows which grades are mixed with each other
Grad-CAM	Shows where the network looked when making a decision

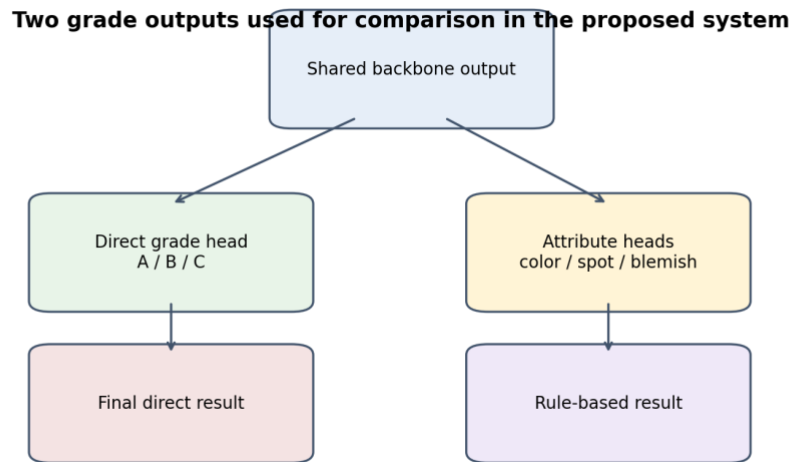


Figure 12: Two grade outputs used for comparison in the proposed system

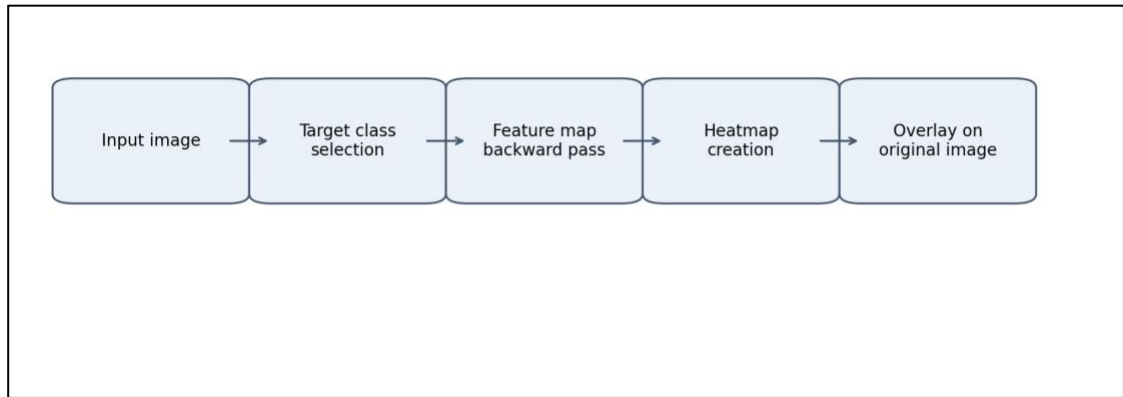


Figure 13: Explainability flow used to generate Grad-CAM visual evidence

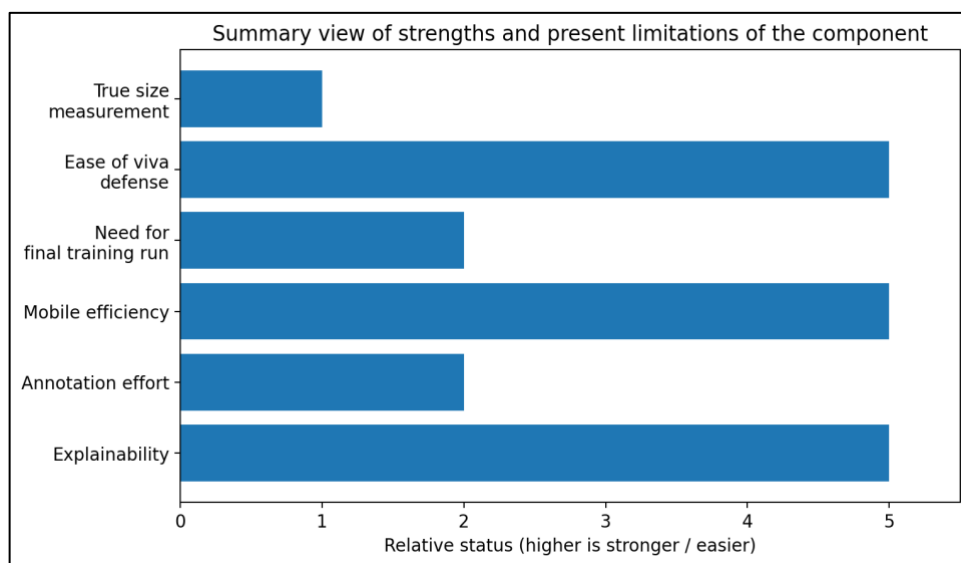


Figure 14: Summary view of strengths and present limitations of the component

3.4 Summary of each student's contribution

My individual contribution in this project is the design and implementation planning of the fruit quality grading module, with mango used as the case study fruit. This includes refining the problem statement for the component, redesigning the dataset structure, defining the new label rubric, planning the preprocessing pipeline, selecting the final backbone, defining the multi-head output structure, and preparing the evaluation and reporting flow.

I also connected the upgraded design to the earlier working baseline code. This is important because it shows that the report is not only a theory document. The component grows from an existing implementation that already used MobileNetV3, stratified splitting, augmentation, class balancing, and saved model exports [18]. My work improves that baseline into a more explainable and academically defensible grading module.

Another key contribution is the way the component is documented. The report explains the full flow in simple English, aligns the model with the thesis format, and clearly marks what belongs to the individual component and what belongs to the larger group system. This makes the work easier for supervisors, panel members, and later developers to understand.

Finally, my contribution includes the creation of the annotation rubric, the report tables, the figure set, the implementation notes, the threshold logic, the saved output structure, and the support material in the appendices. Table 3.5 summarizes these contributions in a short form.

Table 20: Summary of the contribution of this student

Contribution area	Work completed in this component
Problem refinement	Focused the individual work on explainable mango grading
Dataset redesign	Moved from folder-only labels to a richer CSV structure
Annotation rubric	Defined grade, color, spot, and blemish label rules
Preprocessing design	Specified cleanup, crop refinement, resize, and augmentation flow
Model architecture	Selected MobileNetV3-large shared backbone with four heads
Decision logic	Defined direct and rule-based final grade paths
Testing and outputs	Defined metrics, saved reports, exports, and Grad-CAM flow
Documentation	Prepared the full individual summary report and support appendices

4. CONCLUSION

This individual summary report presented the design of an explainable mango quality grading component for the larger FreshRoute project. The report showed why a plain grade-only classifier was not enough and why the dataset needed to be enriched with explicit attribute labels. The final design uses MobileNetV3-large with four heads: color stage, spot severity, blemish severity, and final grade. This makes the grading task easier to explain and easier to defend.

The main value of the component is that it connects human grading logic with deep learning in a simple and practical way. The system does not stop at a black-box class label. It tries to say what visible qualities are driving the final grade. This is important in an academic setting and will also be important in any future real-world deployment.

The report also kept the component scope honest. It focuses only on visible single-image grading. It does not claim to measure physical size from an uncalibrated image or to judge internal fruit quality. It also leaves before-and-after complaint comparison for a later stage of the wider project. This limited but clear scope is one of the strengths of the work.

In conclusion, the fruit quality grading component is now technically stronger, better aligned with the thesis requirements, and more suitable for viva defense. The next practical step is to complete the final annotation pass, run the full training and evaluation pipeline on the frozen dataset, and then plug the grading output into the later complaint validation workflow of the full FreshRoute system.

Table 21: Key conclusions and future improvement directions

Area	Conclusion / future direction
Dataset	The grading task is stronger when attribute labels are added
Model	A multi-head MobileNetV3 design gives clearer outputs than a plain grade-only model
Explainability	Rule-based comparison and Grad-CAM improve understanding
Future work	Complete the final training run and connect the grading module to complaint validation

REFERENCES

- [1] S. R. Dayarathna, "FreshRoute - Smart Fruit Supply Chain System to Reduce Waste and Eliminate Middlemen in Sri Lanka: Project Proposal Report," Sri Lanka Institute of Information Technology, Aug. 2025.
- [2] J. M. Naranjo-Torres et al., "A review of deep learning in agriculture: Techniques and applications," *Computers and Electronics in Agriculture*, vol. 178, 2020.
- [3] Y. Li, C. Y., and J. Wang, "Apple defect detection using deep convolutional networks," *Computers and Electronics in Agriculture*, vol. 167, 2019.
- [4] C. W. Jayasundara and K. Fernando, "Traceability systems for Sri Lanka's fruit exports: Challenges and opportunities," *Tropical Agricultural Research*, vol. 32, pp. 67-78, 2021.
- [5] K. Patel, S. Raman, and V. Shah, "Mango ripeness classification using GLCM and SVM," *International Journal of Computer Applications*, vol. 182, no. 5, pp. 34-39, 2018.
- [6] P. H. Silva and F. Silva, "Texture-based pineapple spoilage detection using Random Forests," *Sri Lanka Journal of Food and Agriculture*, vol. 6, no. 1, pp. 55-62, 2020.
- [7] Sri Lanka Institute of Information Technology, "Guidelines on Documentation and Submission of Dissertations," *Final Thesis Guideline*, 2026.
- [8] S. Perera and S. D. Perera, "Complaint validation using metadata in e-commerce platforms," *Journal of Information Systems Research*, vol. 9, no. 1, pp. 22-34, 2021.
- [9] A. Singh, K. V., and S. R., "Securing mobile applications using in-app camera enforcement," *International Journal of Computer Applications*, vol. 184, no. 36, pp. 19-25, 2022.
- [10] A. Howard et al., "Searching for MobileNetV3," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2019, pp. 1314-1324.
- [11] R. R. Selvaraju et al., "Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization," in *Proceedings of the IEEE International Conference on Computer Vision*, 2017, pp. 618-626.
- [12] C. Zauner, "Implementation and benchmarking of perceptual image hash functions," *Master's thesis*, Upper Austria University of Applied Sciences, 2010.
- [13] J. Deng et al., "ImageNet: A large-scale hierarchical image database," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2009, pp. 248-255.

[14] Uploaded baseline grading code, "model.py," current prototype training script using MobileNetV3-large, stratified split, augmentation, and saved exports.

GLOSSARY

Table 22: Glossary of key terms used in the report

Term	Meaning in this report
Attribute head	A model output branch that predicts one visible quality attribute
Background cleanup	A simple image operation that reduces irrelevant scene information
Blemish	A visible peel mark such as a scar, rough patch, or abrasion
Class imbalance	A condition where some labels have more images than others
Color stage	The visible peel color class: yellow, mixed, or greenish
Grad-CAM	A heatmap method used to explain which region influenced a prediction
Multi-head model	One shared backbone with several output branches
Rule-based grading	Final grade derived from predicted attributes using simple rules
Spot severity	The visible amount and strength of dark spots on the peel
Transfer learning	Fine-tuning a model that already learned from a large dataset

The glossary is included because several terms in deep learning, image processing, and model explanation may be read by both technical and non-technical reviewers. Keeping short definitions in one place makes the report easier to follow.

APPENDICES

Appendix A Annotation rubric

This appendix provides the detailed rubric used for annotation. The rubric is the most important part of the dataset redesign because it turns the label process into a rule-based activity instead of a personal guess. For every image, the annotator should first look at the visible peel color. Then the annotator should look for spots. After that, the annotator should check for blemishes, scratches, scars, and damaged patches. Finally, the annotator assigns the final grade based on the combined visible condition.

Grade A is used when the fruit is mostly yellow and has no severe visual defects. A few very tiny marks can still appear in normal fruit images, but the fruit should still look premium. Grade B is used when the fruit is acceptable but not premium. This often means mixed color or mild surface defects. Grade C is used when the fruit clearly has green dominance, strong ripeness unevenness, or strong visible damage such as severe spots or severe blemishes.

Color stage is a simpler label than grade. A fruit may be yellow, mixed, or greenish. Spot severity is based on the number and visual size of dark spots. Blemish severity is based on visible scars, scratches, rough patches, and abrasion-like damage. Optional defect flags can be used when the image clearly shows specific signs such as black spot, scratch, or discoloration. If the image is too unclear to judge, it should be moved to a review or reject folder rather than forced into a label.

Table 23: Detailed annotation rubric checklist

Item	What the annotator should check
Peel color	Is the fruit mostly yellow, mixed, or greenish?
Dark spots	Are there no spots, a few small spots, or many / large spots?
Blemishes	Are there scars, scratches, rough patches, or abrasion marks?
Overall look	Does the fruit look premium, acceptable, or clearly low quality?
Image quality	Is the photo clear enough for a reliable label?

Appendix B CSV format

This appendix gives the CSV file format used by the component. The grade label is kept because it is still needed for the final grade head. The new attribute columns are added so that the model can learn explainable grading cues. Optional defect flags are kept as binary values because they are simple to store and can later support extra analysis.

The label file should be saved in UTF-8 CSV format. The image path should be relative to the image root folder so that the same file can be reused on different machines. The annotator and review status columns are optional but recommended because they make the label history easier to manage. If a later correction is made, the CSV file can be updated without having to rebuild the full folder structure.

```
image_path,grade,color_stage,spot_severity,blemish_severity,black_spot,scratch,discoloration
GradeA/img001.jpg,A,yellow,none,none,0,0,0
GradeA/img002.jpg,A,yellow,mild,none,0,0,0
GradeB/img101.jpg,B,mixed,mild,mild,1,0,0
GradeC/img201.jpg,C,greenish,severe,severe,1,1,1
```

Appendix C Folder structure and outputs

The implementation is easier to maintain when the folder structure is clear. The report recommends separate folders for images, label files, split files, outputs, and excluded images. The outputs folder should then contain subfolders for the baseline model, the multi-head model, confusion matrices, Grad-CAM results, exported models, and threshold files. This structure mirrors the saved artifacts explained in Chapter 2 and Chapter 3.

A clean folder structure also helps dissertation writing. When the training script saves files into fixed locations, the student can quickly find the latest confusion matrix, the latest exported model, or the latest Grad-CAM example. This saves time during testing, figure preparation, and supervisor review.

```
project/
  data/
```

```

images/GradeA, GradeB, GradeC
labels.csv
splits/train.csv, val.csv, test.csv
outputs/
baseline/
multitask/
confusion_matrices/
gradcam/
exports/

```

Appendix D Training settings and implementation notes

This appendix records the training settings used in the current baseline code and the planned upgrade for the multi-head model. The baseline already uses a 224 x 224 image size, weighted sampling, a small learning-rate grid, a class-weighted cross-entropy loss, and early stopping. These settings are reasonable for a student project because they balance simplicity and usefulness.

The upgraded model keeps the same general training style and adds the combined loss for the four heads. The recommended starting point is to keep the grade loss weight at 1.0 and the three attribute loss weights at 0.7 each. The final values can be tuned later, but the starting point is easy to explain and matches the design logic of the component.

Table 24: Loss components in the multi-head model

Loss term	Weight	Reason
grade_loss	1.0	Final grade is the main output
color_loss	0.7	Color stage supports explainability
spot_loss	0.7	Spot severity supports explainability
blemish_loss	0.7	Blemish severity supports explainability